



Projet final - Linux Dev.

Ce projet représente l'évaluation finale du cours de Linux orienté Développeurs.

Le but de ce projet est de créer et configurer un environnement Linux à des fins de développement. Vous aurez besoin de toutes les connaissances acquises pendant le cours.

La date de rendu du projet est indiquée sur l'intranet myGES dans le projet pédagogique correspondant. Chaque étape se compose d'un barème défini. Le mode de rendu est indiqué dans la partie mise en place et spécifiquement parfois dans chacun des exercices.

Il est fortement conseillé de lire le sujet une première fois en entier pour voir certains liens entre les différents exercices.

Le projet fera l'objet d'une soutenance pour présenter l'environnement et l'aboutissement de celui-ci. La démonstration devra être structurée.

Toute tentative de triche sera bien évidemment sanctionnée.

Mise en place (2 pts)

Créer une machine virtuelle avec au moins 2Go de RAM et 20Go de disque dur. Cette machine devra impérativement être basée sur une image **Ubuntu Server 24.04+** et non Desktop.

Voici quelques paramètres à configurer au moment de l'installation :

- Langue système : anglais ;
- Utilisateur : `modo` ;
- Mot de passe : `qwerty123` ;
- Nom du serveur : `linux-exam-vm` .

Il est nécessaire de mettre en place un fichier au coeur de l'environnement pour définir l'appartenance de celui-ci :

- Créer dans le dossier utilisateur de `modo` un fichier `authors.txt` avec pour contenu la liste des étudiants au format suivant : une astérisque (`*`), suivie d'un espace puis des 6 premières lettres maximum du nom de l'étudiant, d'un underscore, de la première lettre du prénom de l'étudiant et enfin d'un **retour à la ligne** ⇒ `* martin_m`

Enfin, il faut réactiver l'utilisateur `root` . Si vous devez préciser des informations concernant l'utilisateur `root` , vous pouvez les mettre dans un fichier `root.txt` à rendre également.

Pour le rendu, la machine virtuelle devra être sous format OVA ou équivalent. En fonction de vos capacités d'upload, vous pouvez rendre un fichier contenant le lien où télécharger la machine virtuelle.

Rendu : `vm-url.txt` , [`root.txt`]

SSH (1 pt)

La machine devra être accessible via SSH. L'utilisateur `root` ne doit pas pouvoir se connecter directement à la machine. Vous devrez générer une paire de clef SSH pour l'utilisateur `modo` à intégrer au rendu, le but étant de pouvoir se connecter avec une des clefs.

Rendu : `ssh-keys.zip`

Configuration (1 pt)

Vous devez configurer correctement l'environnement de l'utilisateur `modo` avec les fonctionnalités suivantes :

- Shell par défaut : ZSH ;
- Framework Oh My Zsh : <https://ohmyz.sh/> ;

- Theme Haribo (à installer) pour Oh My Zsh ;

Application web Node.js (2 pts)

Vous devez mettre en place une application web en Node.js accessible depuis l'extérieur via un serveur HTTP Nginx, Apache ou Caddy.

L'application web devra utiliser le CMS **Strapi**. Pour mettre en place cette application, vous pouvez utiliser la documentation suivante :

<https://docs.strapi.io/dev-docs/intro>.

Démonstration et rendu

Vous devez mettre en place des types de contenu en rapport avec un blog contenant des articles et des utilisateurs.

L'url et/ou le port d'accès à l'application devront être inclus au rendu dans un fichier texte.

Rendu : `app-web.txt`

Base de données (3 pts)

Vous devez mettre en place une base de données PostgreSQL en version 16+ avec les identifiants suivants :

- Base de données : `data-db`
- Utilisateur : `dbuser`
- Mot de passe : `gTU1ZwxE92Z77H83a33OZ046`
- L'utilisateur peut se connecter et aussi manipuler complètement la base de données déclarée précédemment.

Il existe différentes façons de faire pour déclarer un nouvel utilisateur, tant que les identifiants et les accès sont correctement respectés, c'est bon.

Accès extérieur

 La base de données doit être **accessible depuis l'extérieur**. Il faut comprendre que via l'adresse IP de la machine virtuelle, je dois pouvoir y accéder, via un outil externe comme DataGrip par exemple.

Schéma

La base de données doit respecter le schéma suivant :

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE events (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  description TEXT,  
  date TIMESTAMP WITH TIME ZONE NOT NULL,  
  location VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE event_participants (  
  id SERIAL PRIMARY KEY,  
  user_id INT NOT NULL,  
  event_id INT NOT NULL,  
  joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE  
);
```

Backup récurrent

Vous devez également mettre en place une tâche cron permettant de faire un backup de la base de données tous les deux jours à 02h04. Les fichiers devront être stockés dans le dossier `/opt/backup/postgresql`.

Serveur SFTP (2 pts)

Vous devez mettre en place un serveur **SFTP** (Secured FTP) sur la machine pour que l'on puisse y manipuler des fichiers. Il doit être cloisonné à un dossier précis du système pour un utilisateur en particulier.

Vous devez utiliser `vsftpd` pour mettre en place ce serveur.

Les identifiants de l'utilisateur en question ainsi que le dossier ciblé doivent être inclus au rendu.

Rendu : `sftp-identifiers.txt`

Pare-feu (2 pts)

Vous devez mettre en place un pare-feu pour bloquer toutes les connexions entrantes sauf celles nécessaires pour votre machine virtuelle.

Scripting

Vous devez obligatoirement utiliser `iptables` pour gérer vos règles de pare-feu. Il est donc impératif de créer un script avec vos règles de façon à ce qu'elles soient persistées entre chaque démarrage de votre machine.

Les règles mises en place seront à **détailler avec des commentaires en anglais** pour justifier leur mise en place.

Rendu : `firewall-script.sh`

Scripting et cron (2 pts)

Vous avez des consignes à respecter pour réaliser différents scripts et cron. Ces scripts doivent être autonomes et utilisables sur une autre machine Linux. Si vous avez besoin de paquets externes, il faut prévoir de les installer au moment de l'utilisation du script.

Jours fériés (1 pt)

Vous devez créer un script permettant de renvoyer le prochain jour férié en France métropolitaine par rapport à la date actuelle de la machine. Vous devez dans un premier temps récupérer les jours fériés en France depuis l'API officielle du gouvernement :

- <https://www.data.gouv.fr/fr/dataservices/jours-feries/>.

Consignes

- Si aujourd'hui est un jour férié, il faut renvoyer celui d'après ;
- Si une erreur de l'API survient, un message d'erreur devra être renvoyé.

Météo (1 pt)

Vous devez créer une tâche cron permettant de récupérer tous les jours la météo du jour et l'afficher dans le message du jour du système.

Vous devez vous baser sur l'API OpenWeatherMap :

- <https://openweathermap.org/api>.

Consignes

- Récupérer la météo du jour tous les jours à 6h ;
 - Température moyenne en journée (°C)
 - Météo du jour (ex: ensoleillé, nuageux, pluie, etc.)
- Modifier le message du jour `motd` en ajoutant les informations précédentes de façon cohérente.

Environnement et services (2 pts)

Vous devez réaliser un service `systemd` pour vérifier l'intégrité d'un environnement de développement Flutter tous les jours.

Scripting

Pour effectuer la vérification d'intégrité, vous devez réaliser un script qui va vérifier si le dossier du SDK est présent et si la commande `flutter doctor` s'est bien passé. À vous d'arriver à un script cohérent qui parse correctement les sorties.

Le script doit écrire dans un fichier de logs spécifique et l'écraser avant chaque vérification. Il doit également préciser la date et l'heure de la vérification.

Service

Le service `systemd` doit exécuter le script tous les jours en tant qu'un utilisateur spécifique à définir.

Bonus

Les fonctionnalités précédentes ne suffisent pas pour avoir la note maximale. Voici une liste d'exemple de bonus à faire pour compléter votre note. D'autres bonus sont possibles et seront évalués a posteriori :

- Configurer docker sur la machine et faire en sorte d'utiliser des conteneurs pour les différents environnements à mettre en place ;
- Scripts utilitaires pour le développement et le déploiement des ressources sur l'environnement ;
- Script pour remplir la base de données avec des données d'exemple cohérentes ;
- Déploiement de votre propre application front qui récupère les données de la base de données ou du CMS mis en place précédemment ;
- Rajouter une notification dans un canal Discord après chaque vérification d'intégrité pour l'exercice "Environnement et services" ;
- ...

Dans votre rendu, si des bonus sont présents, vous devez lister les bonus choisis, même ceux qui ne sont pas présents dans la liste ci-dessus.

Rendu : `bonus-list.txt`